REPORTE DE ACTIVIDAD: IA.C2.A1 ALGORITMOS GENÉTICOS 1

Inteligencia artificial – 8° “A” – Corte #2

Universidad Politécnica de Chiapas
Ingeniería en Desarrollo de Software

Javier Alberto Argüello Tello - 153217

16 de febrero del 2018

Cuatrimestre	Enero – Abril de 2019
Cuatrimestre y grupo	8A
Asignatura	Inteligencia Artificial
Corte	02
Actividad	IA.C2.A1 Algoritmos genéticos 1
Fecha de asignación	2019.02.11
Fecha de entrega	2019.02.14 (Digital) 2019.02.15 (Revisión)
Matrícula	**Nombre**
153217	ARGÜELLO TELLO Javier Alberto

Tabla de contenido

1 Introducción 2
2 Configuración del sistema 2
2.1 Estructura de los archivos 2
2.2 Estructura del código 3
2.3 Etapas 5
2.3.1 Inicialización 5
2.3.2 Evaluación 5
2.3.3 Cruce 6
2.3.4 Mutación 7
2.3.5 Poda 8
2.3.6 Llevar a cabo las iteraciones / Evolucionar 8
3 Resultados 10
3.1 Video que muestra la evolución de la población. 10
3.2 Gráfica donde se visualiza la evolución de la media, el mejor caso y el peor caso para todas las generaciones 14
4 Discusión 14
5 Bibliografía 15

1 Introducción[1]

Los Algoritmos Genéticos son métodos adaptativos que pueden usarse para resolver problemas de búsqueda y optimización. Están basados en el proceso genético de los organismos vivos.

Usan una analogía directa con el comportamiento natural. Trabajan con una población de individuos, cada uno de los cuales representa una solución factible a un problema dado. A cada individuo se le asigna un valor o puntuación, llamado "fitness", relacionado con la bondad de dicha solución. En la naturaleza esto equivaldría al grado de efectividad de un organismo para competir por unos determinados recursos. Cuanto mayor sea el fitness de un individuo al problema, mayor será la probabilidad de que el mismo sea seleccionado para reproducirse, cruzando su material genético con otro individuo seleccionado de igual forma. Este cruce producirá nuevos individuos, descendientes de los anteriores, los cuales comparten algunas de las características de sus padres. Cuanto menor sea la adaptación de un individuo, menor será la probabilidad de que sea seleccionado para la reproducción y, por tanto, de que su material genético se propague en sucesivas generaciones. De esta manera se produce una nueva población de posibles soluciones, la cual reemplaza a la anterior y verifica la interesante propiedad de que contiene una mayor proporción de buenas características en comparación con la población anterior. Así a lo largo de las generaciones las buenas características se propagan a través de la población.

El poder de estos proviene del hecho de que se trata de una técnica robusta, y pueden tratar con éxito una gran variedad de problemas provenientes de diferentes áreas, incluyendo aquellos en los que otros métodos encuentran dificultades. Si bien no se garantiza que este algoritmo encuentre la solución óptima del problema, existe evidencia empírica de que se encuentran soluciones de un nivel aceptable, en un tiempo competitivo con el resto de algoritmos de optimización combinatoria.

El objetivo de esta práctica consiste en implementar un algoritmo genético para hallar el máximo de una función en el espacio, de la forma $f(x, y)$; con rangos definidos para cada eje.

2 Configuración del sistema

Se usó el lenguaje de programación Python, en su versión 3.6.5. Además de las siguientes librerías externas:

- ***numpy:*** Para acceder a funciones de generación de valores aleatorios y funciones matemáticas varias.
- ***scipy:*** Para acceder a una función CDF.
- ***matplotlib:*** Para crear gráficas.
- **cv2:** Para crear videos.

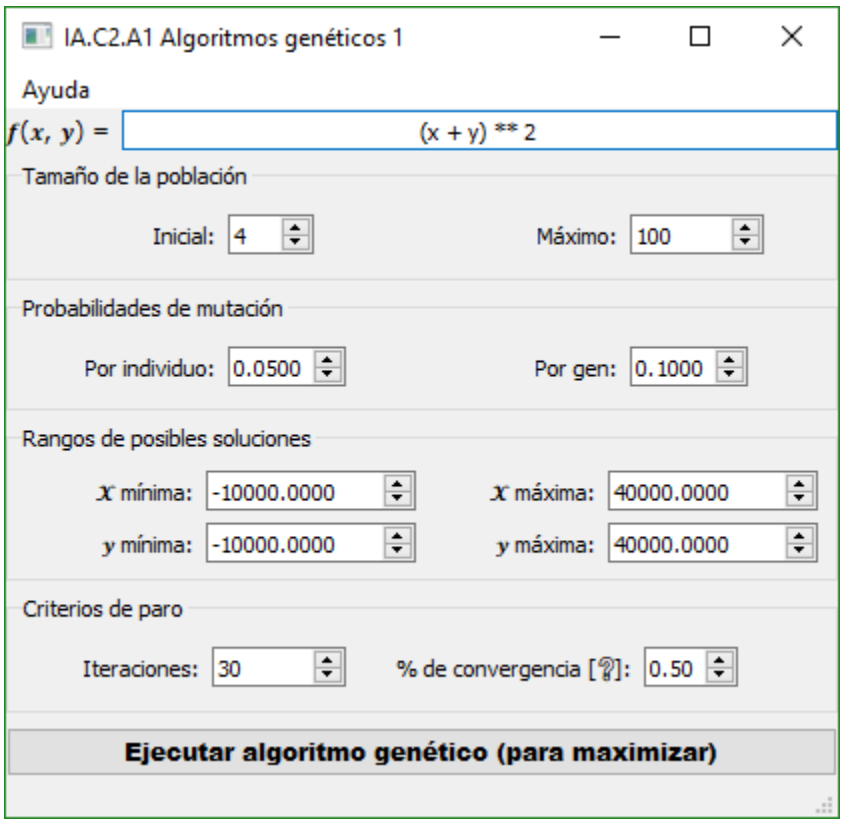
2.1 Estructura de los archivos

La aplicación tiene una estructura sencilla, que se compone de los siguientes archivos:

- ***GUI.ui:*** Contiene la información de la vista/interfaz. Está en un formato que sólo Qt puede entender.
- ***Ejecutable.pyw:*** Es el archivo ejecutable, actúa como el controlador de la interfaz.
- ***AlgGen.*****py:** Es un módulo que contiene todo el código relacionado con el algoritmo genético. Puede ser reutilizado en otras aplicaciones. El código más importante se encuentra dentro de éste.

[1] Sección basada principalmente en la referencia 1.

La interfaz gráfica tiene el siguiente aspecto:



Cabe destacar, que la función debe ser escrita en un formato que Python entienda. Por ejemplo, la función escrita en la captura de pantalla representa:

$$f(x, y) = (x + y)^2$$

2.2 Estructura del código

'AlgGen.py', el script más importante, se realizó siguiendo el paradigma orientado a objetos. El código es más fácil de entender y mantener, pero es un poco más costoso computacionalmente, en comparación a haber realizado una sola gran función. El autor consideró que dichos beneficios valen la pena, el costo extra.

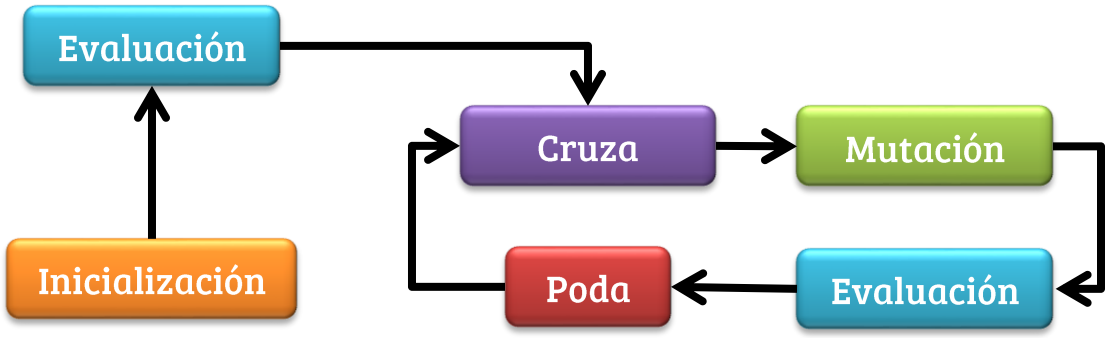
Se tienen las siguientes clases:

- **"Individuo"**: Representa a un individuo de la población. Sus atributos principales son un cromosoma (una cadena de caracteres '0' y '1'), un valor de fitness y una calculadora de fitness (la siguiente clase). Todos los individuos de una misma población tienen, la misma instancia de la calculadora de fitness. El tamaño del cromosoma es una constante, la cual, está definido en esta clase; su valor es de 32. El constructor recibe una instancia de la calculadora y de manera opcional, el cromosoma (no se manda cuando se está creando una población inicial; y si, cuando se está creando un hijo). Tiene los siguientes métodos:
  - **Un getter y un setter para el cromosoma.** Cuando se llama al último, el valor de fitness se actualiza haciendo uso de la calculadora de fitness.
  - **Un getter para el valor de fitness.**

  - **'reproducirse'**, el cual recibe otro individuo para realizar un apareamiento y generar dos hijos o individuos nuevos.
  - **'calcular_x_y':** Un método que llama al método del mismo nombre de la calculadora de fitness.
- **"CalculadoraFitness":** Las instancias de esta clase permiten calcular el fenotipo y el valor de fitness (*que nos permite analizar qué tan bueno/apto es un individuo*), en base a un cromosoma. Para ello, en el constructor, requiere de la definición de la *función de fitness* o *función de adaptación*, (la función que se quiere maximizar) y los intervalos de posibles soluciones en $x \in [a, b]$ y $y \in [c, d]$. Tiene los siguientes métodos:
  - **'calcular_x_y':** Recibe un cromosoma y calcula su fenotipo, es decir, los valores de 'x' y 'y', dentro de los rangos de posibles soluciones.
  - **'calcular_fitness':** Recibe un cromosoma y calcula su valor de fitness. En otras palabras, obtiene los dos valores del fenotipo del cromosoma (usando el método anterior) y evalúa la función de fitness con ellos. *En este método se realiza la etapa de evaluación.*
- **"PoblacionAlgGen":** Representa a una población capaz de realizar todas las etapas del algoritmo genético. Esta es la clase principal y la única con la que debe interactuar el usuario. Su atributo principal es la población (una lista de objetos de tipo 'Individuo'). En su constructor recibe todos los valores de los campos de la interfaz; crea una calculadora de fitness y una población inicial (*realiza la etapa de inicialización*). Tiene los siguientes métodos:
  - **'cruzar':** Lleva a cabo la etapa de cruce entre los individuos, resultando en un aumento de la población.
  - **'mutar':** Lleva a cabo la etapa de mutación, en la que algunos individuos nuevos de la generación actual, elegidos al azar, cambian algunos de sus genes, elegidos al azar. Se tienen probabilidades diferentes para cada una de estos dos.
  - **'podar':** Se eliminan a los peores individuos, aquellos con los fitnesses más bajos.
  - **'evolucionar', 'actualizar_salida_video' y 'actualizar_salida_grafica':** Crea generaciones nuevas que reemplazan a las previas, ejecutando los primeros tres métodos de forma iterativa. Además, genera ciertas salidas, que se describen en la sección 2.3.6.

2.3 Etapas

El ciclo de vida del algoritmo es el siguiente:

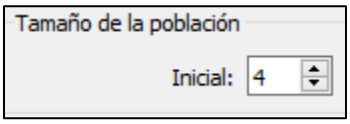


2.3.1 Inicialización

Consiste en crear una población inicial. El tamaño de la población está determinado por el usuario.

Cada individuo nuevo tiene un cromosoma tamaño 32, hecho al azar.

Para esta etapa se usa el dato del siguiente control:



En el código, se lleva a cabo en los constructores de *'PoblacionAlgGen'* e *'Individuo'*.

2.3.2 Evaluación

Consiste en obtener el valor de fitness de cada individuo. Esta es una métrica que nos permite saber, dentro de la población actual, qué individuos resuelven mejor el problema que, en este caso, consiste en maximizar una función.

Para ello se realiza lo siguiente:

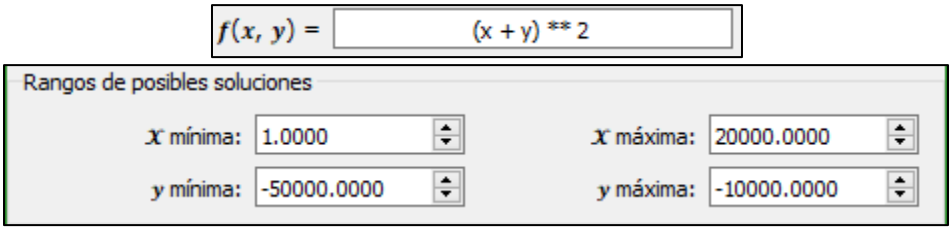
1. Se calcula el fenotipo del individuo (valores 'x' y 'y'), usando el cromosoma.
   a. El cromosoma se parte a la mitad. Cada mitad tiene un tamaño de 16.
   b. Cada mitad se convierte, de una cadena binaria, a un valor entero.
   c. Se obtienen los valores de 'x' y 'y' usando las siguientes fórmulas:

$$x = a + (m_1 * \Delta_x) \qquad y = c + (m_2 * \Delta_y) \qquad \Delta_x = \frac{abs(b-a)}{M} \qquad \Delta_y = \frac{abs(d-c)}{M}$$

   Donde:
   - 'm_1' y 'm_2' son la primera y segunda mitad, respectivamente.
   - 'a' y 'b' son el inicio y el final, respectivamente, del intervalo de posibles soluciones en el eje 'x'.
   - 'c' y 'd' son el inicio y el final, respectivamente, del intervalo de posibles soluciones en el eje 'y'. Tanto estos, como los anteriores, son definidos por el usuario.
   - 'M' es el valor máximo que puede tomar una mitad. En este caso, '1111111111111111' convertido a entero: 65535.

2. Se usan los valores de 'x' y 'y' para evaluar la función fitness $f(x, y)$, la cual, es definida por el usuario y corresponde a la función que se quiere maximizar. El resultado será el valor de fitness para el individuo actual.

Para esta etapa se usan los datos de los siguientes controles:



En el código, se lleva a cabo:

- Al crear un individuo en la etapa de inicialización o cruza; el constructor manda a llamar a '*Individuo. set cromosoma*' que, a su vez, manda a llamar a '*CalculadoraFitness. calcular fitness*' que, a su vez, manda a llamar a '*CalculadoraFitness. calcular x y*'.
- Al mutar un individuo, se mandan a llamar '*Individuo. set cromosoma*', '*CalculadoraFitness. calcular fitness*' y '*CalculadoraFitness. calcular x y*'; para actualizar su valor de fitness.

2.3.3 Cruce

Consiste en hacer que parejas de individuos al azar se reproduzcan y hagan que la población crezca.

Para ello, se realiza el siguiente proceso:

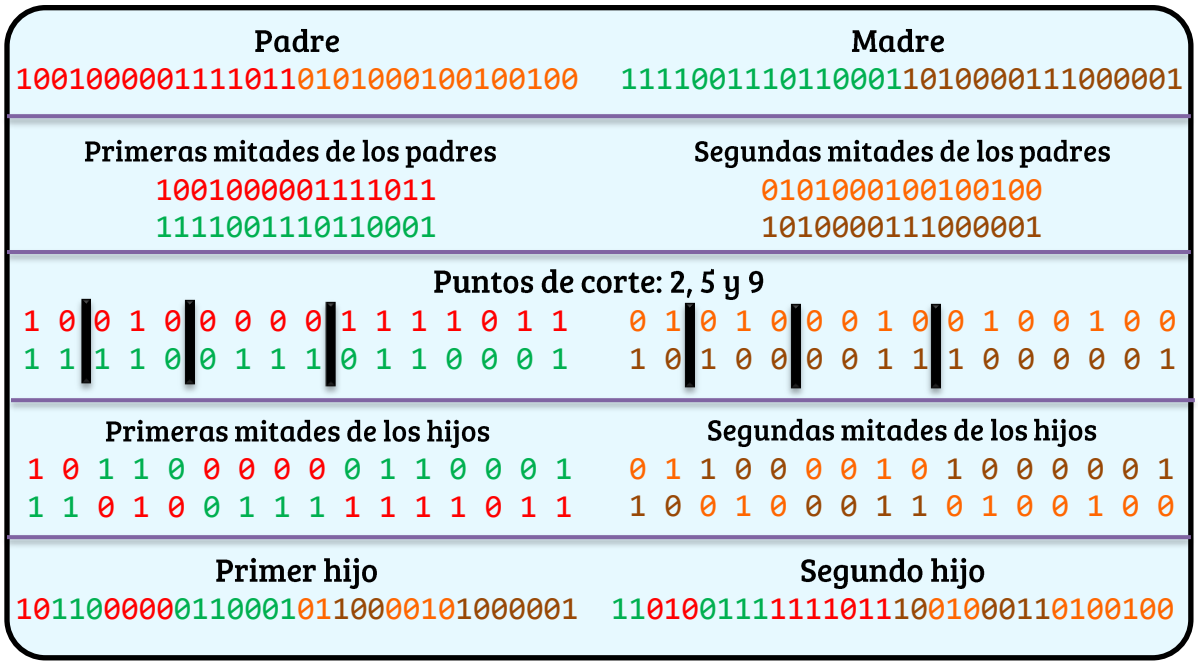
1. Se ordenan los individuos por su fitness.
2. Se obtiene la probabilidad de que cada individuo se reproduzca.
    a. Se calculan, la media y la desviación estándar de los valores de fitness.
    b. Se ejecuta la función CDF, con siguientes parámetros: valores de fitness, media y desviación estándar. Ésta retorna la probabilidad acumulada en una distribución normal, para cada individuo. Mientras más alto sea el fitness de un individuo, más alta será su probabilidad.
3. Se itera por la población.
    a. Se decide si el individuo actual se reproduce, usando su respectiva probabilidad.
    b. Si se decidió que sí, se itera por los pretendientes: aquellos individuos que tengan una menor probabilidad de reproducirse que la del individuo actual.
        i. Se decide si el pretendiente actual se reproduce con el individuo actual, usando la probabilidad de reproducción del primero.
        ii. Si se decidió que sí, ambos tienen dos hijos, los cuales pasan por el proceso de mutación y finalmente, pasan a ser parte de la población (cabe destacar que los hijos no podrán reproducirse, sino hasta la siguiente generación).

Para cruzar dos individuos, se realiza el siguiente proceso:

1. Se decide al azar, cuántos puntos de corte habrá: entre 1 y 5.
2. Se deciden cuáles van a ser los puntos de corte (las posiciones del cromosoma) al azar.
3. Los cromosomas de los padres se dividen en dos, cada uno. Se tienen 4 mitades: 2 primeras y 2 segundas.
4. Cada una de las mitades se trocea usando los puntos de corte.

5. Las primeras mitades y las segundas mitades se mezclan por separado.
    a. Se tienen dos mitades que se van a unir, ya sean las primeras o las segundas. Se crean dos primeras mitades de cromosomas de los dos hijos, intercalando los trocitos. Por ejemplo:
        i. La mitad del cromosoma del primer hijo tendrá el primer trozo de la mitad del cromosoma del padre, el segundo trozo de la mitad del cromosoma de la madre, el tercer trozo de la mitad del cromosoma del padre y así, sucesivamente.
        ii. La mitad del cromosoma del segundo hijo tendrá el primer trozo de la mitad del cromosoma de la madre, el segundo trozo de la mitad del cromosoma del padre, el tercer trozo de la mitad del cromosoma de la madre y así, sucesivamente.
6. Las mitades de los cromosomas cada uno de los hijos se unen. Y luego, se usan para crear individuos, que pasarán a la fase de mutación y finalmente, a formar parte de la población.

El procedimiento recién explicado es complicado de entender, así que a continuación se presenta un ejemplo visual:



En el código, se lleva a cabo en '*PoblacionAlgGen.cruzar*' e '*Individuo. reproducirse*'.

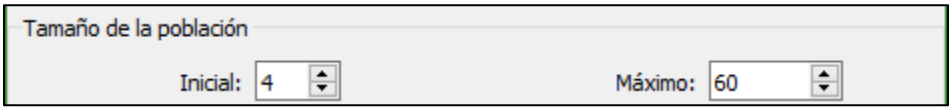
2.3.4 Mutación

Consiste en alterar ciertos genes de los individuos nuevos de la generación actual (los recién nacidos). Se hace porque existe la posibilidad de que dicha alteración, mejore al individuo.

Para ello se realiza el siguiente procedimiento:

1. Se itera por los individuos recién nacidos. En cada iteración, se decide al azar si el individuo actual va a mutar, usando una probabilidad definida por el usuario.
2. Por cada individuo que se decidió mutar, se itera por sus genes (elementos del cromosoma) y se decide al azar si se va a aplicar la mutación al gen actual, usando otra probabilidad definida por el usuario (diferente a la del punto anterior).
3. Cada gen que se decidió mutar, pasa a ser '1' si vale '0' y viceversa.

Para esta etapa se usan los datos de los siguientes controles:



En el código, se lleva a cabo en '*PoblacionAlgGen.cruzar*' que, a su vez, manda a llamar '*PoblacionAlgGen.mutar*'.

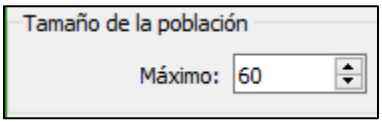
2.3.5 Poda

Consiste en eliminar a los peores individuos (aquellos con el valor de fitness más bajo) de la generación actual.

Para ello se realiza el siguiente procedimiento:

1. Se ordena la población por su fitness.
2. Se evalúa si ya se ha superado el límite de individuos o la población máxima, definida por el usuario.
    a. Si se superó el límite, se elimina al exceso de individuos.
    b. Si no se superó el límite, se elimina a ⅓ de los individuos.

Para esta etapa se usa el dato del siguiente control:



En el código, se lleva a cabo en '*PoblacionAlgGen.podar*'.

2.3.6 Llevar a cabo las iteraciones / Evolucionar

Aunque no es una etapa como tal, este es el proceso que controla las etapas que se ejecutan de manera cíclica.

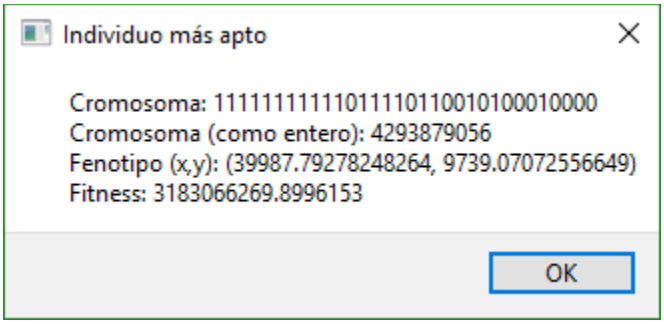
Hace que la población avance por una cierta cantidad de generaciones o iteraciones, mientras que va guardando cierta información de cada generación para generar estas salidas:

- Un video donde se visualiza la evolución a través de las generaciones, de la distribución de los individuos en el plano xy.
- Una gráfica donde se visualiza la evolución de los valores de fitness: la media, el mejor caso y el peor caso; para todas las generaciones, antes de la poda.

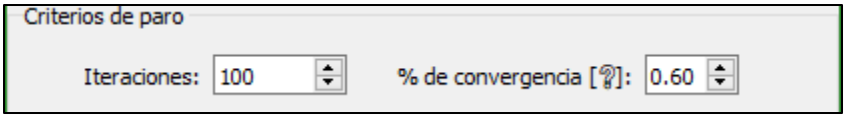
Para ello se realiza el siguiente procedimiento:

1. Se crean los almacenes para la información de las salidas: un registro de medias, mejores y peores casos; así como un escritor de video. El video se guarda con una resolución de 1280x720 y con una velocidad de 4 fotogramas por segundo.
2. Se guarda la información de la generación inicial para generar las salidas (la media, el mejor y peor caso; así como un frame del video).
3. Se realiza un ciclo con una cantidad de iteraciones especificada por el usuario.
    a. Se realizan las etapas de cruce, mutación y evaluación.
    b. Se guarda la información necesaria para generar la gráfica.
    c. Se realiza la poda.

   d. Se guarda un frame del video, correspondiente a la generación actual.
   e. Se evalúa si el mejor valor de fitness se encuentra repetido en cierto porcentaje de la población; si es así, se dice que ya se ha convergido a la solución y el resto de iteraciones queda cancelado. Este porcentaje se lama “de convergencia” y es establecido por el usuario.
4. El escritor de video se cierra, dejando el video guardado en una ubicación especificada por el usuario.
5. Se usan los datos de las medias, los mejores y los peores casos, para generar y mostrar la gráfica.
6. Se retorna la información del mejor individuo de la generación más reciente, para mostrárselo al usuario de este modo:



Para este procedimiento, se usan los datos de los siguientes controles:

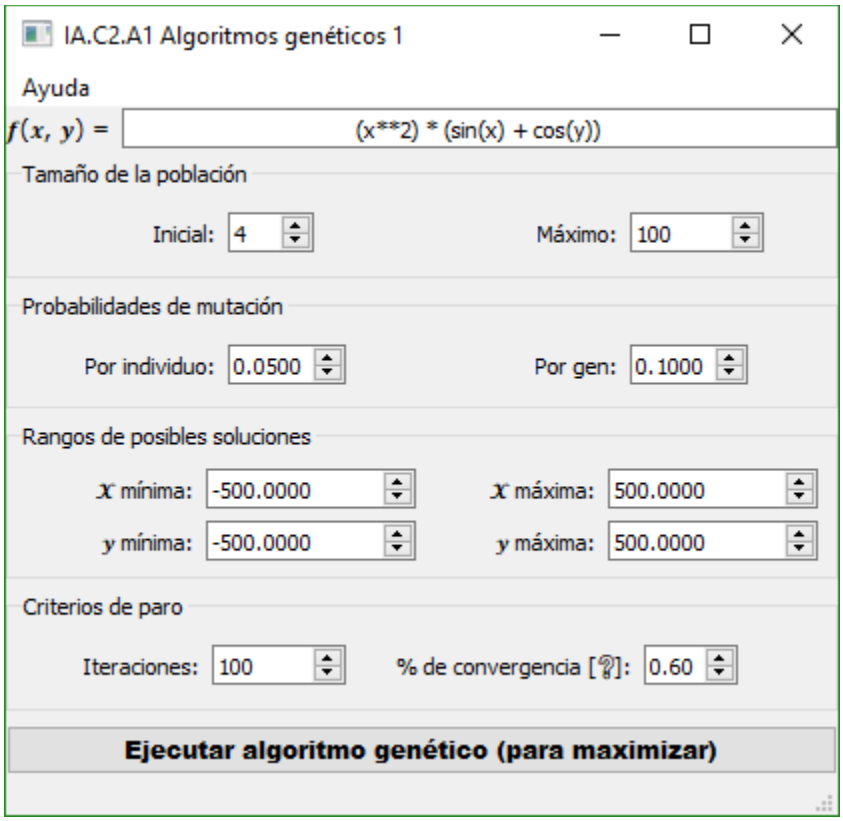


3 Resultados

Se buscaron los parámetros (valores de 'x' y 'y') que dan un mayor resultado para la siguiente función:

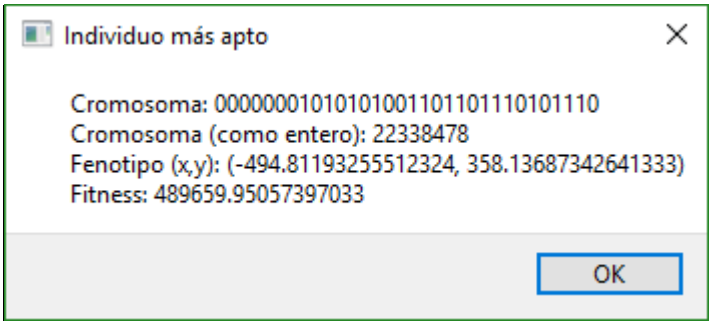
$$f(x,y) = x^2(\sin(x) + \cos(y))$$

La configuración que se usó, se puede apreciar en la siguiente captura de pantalla:



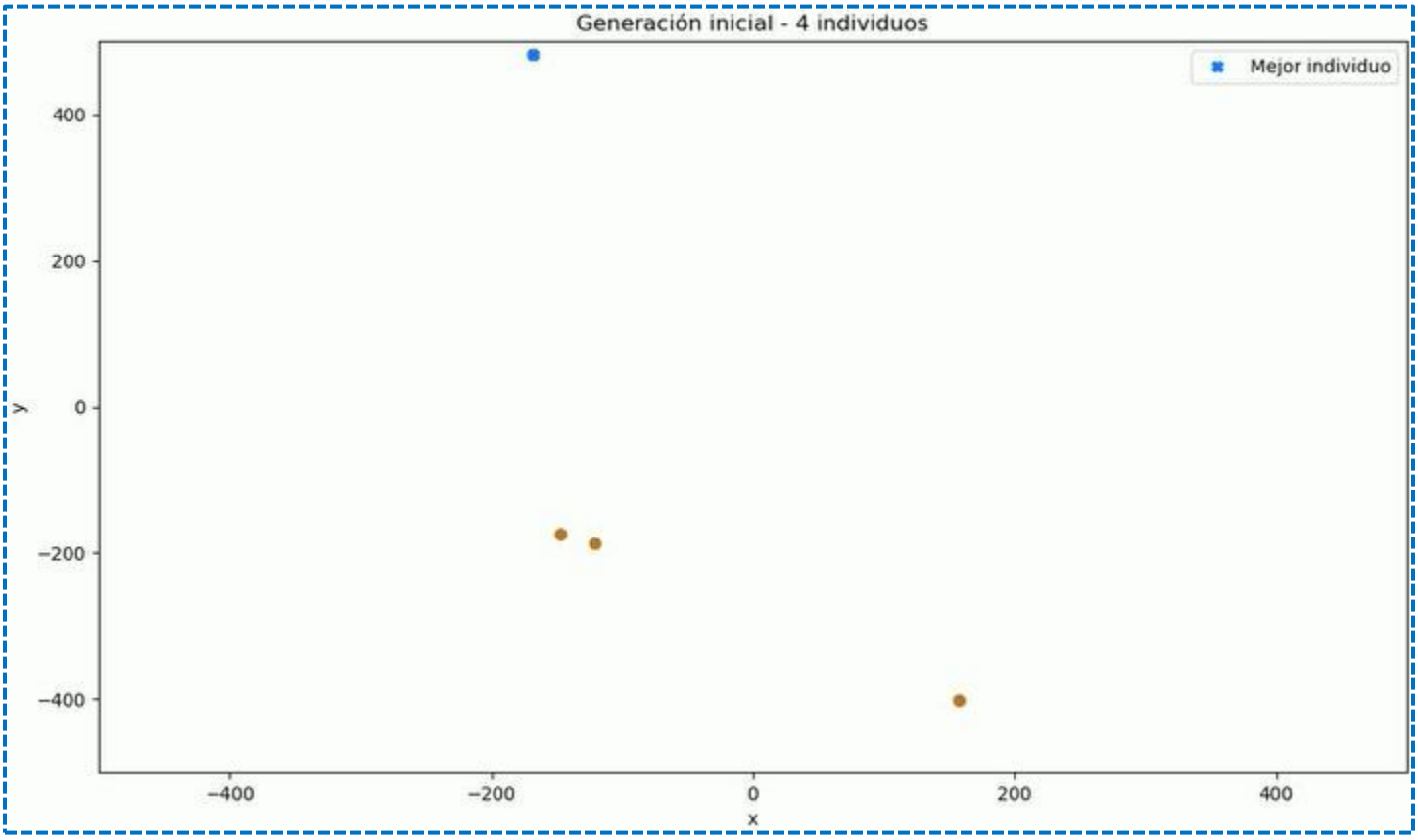
Al terminar la última iteración, el mejor individuo de la población tuvo el siguiente fenotipo y valor de fitness:

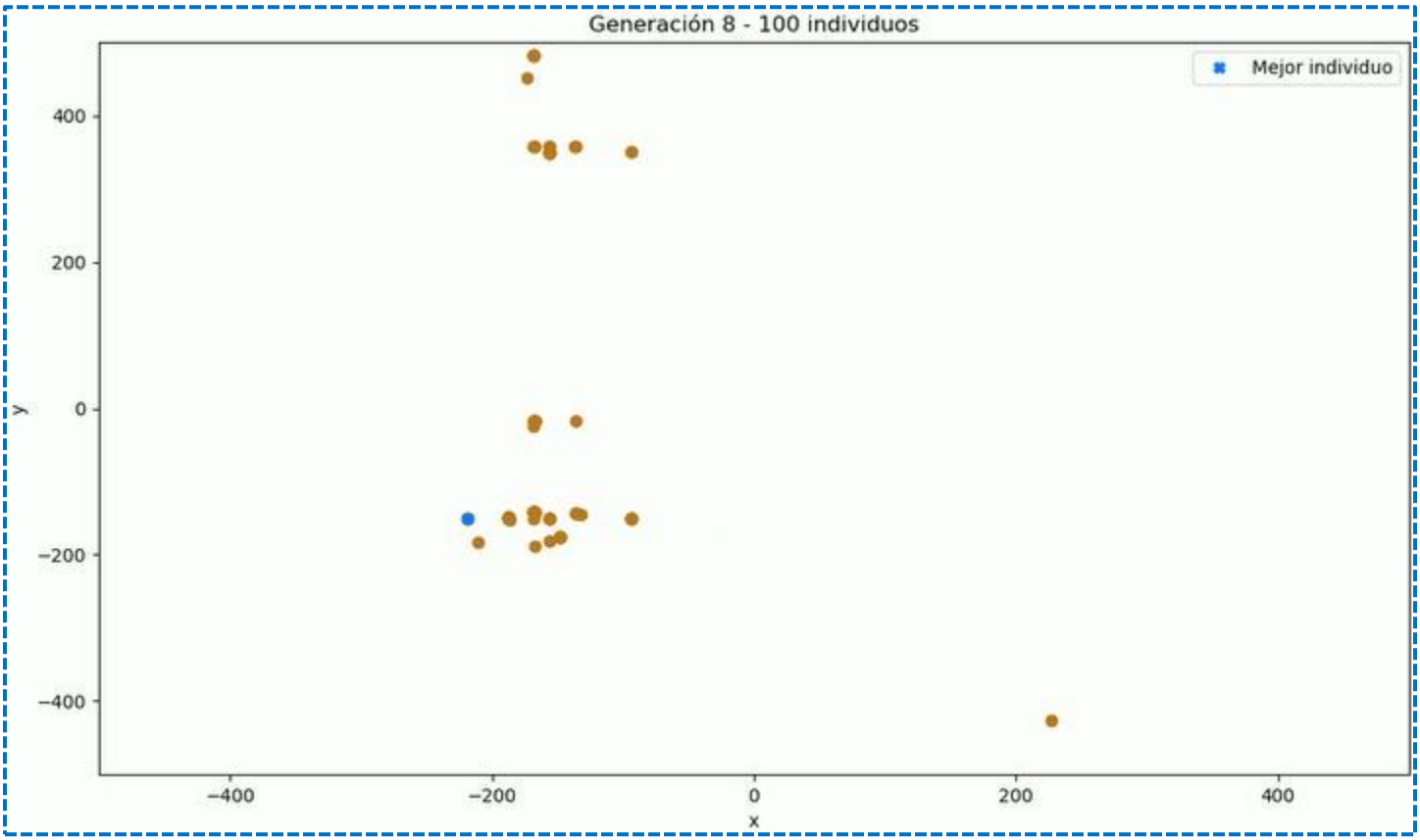
$$x = -494.81193255512324 \qquad y = 358.13687342641333 \qquad f(x,y) = 489659.95057397033$$

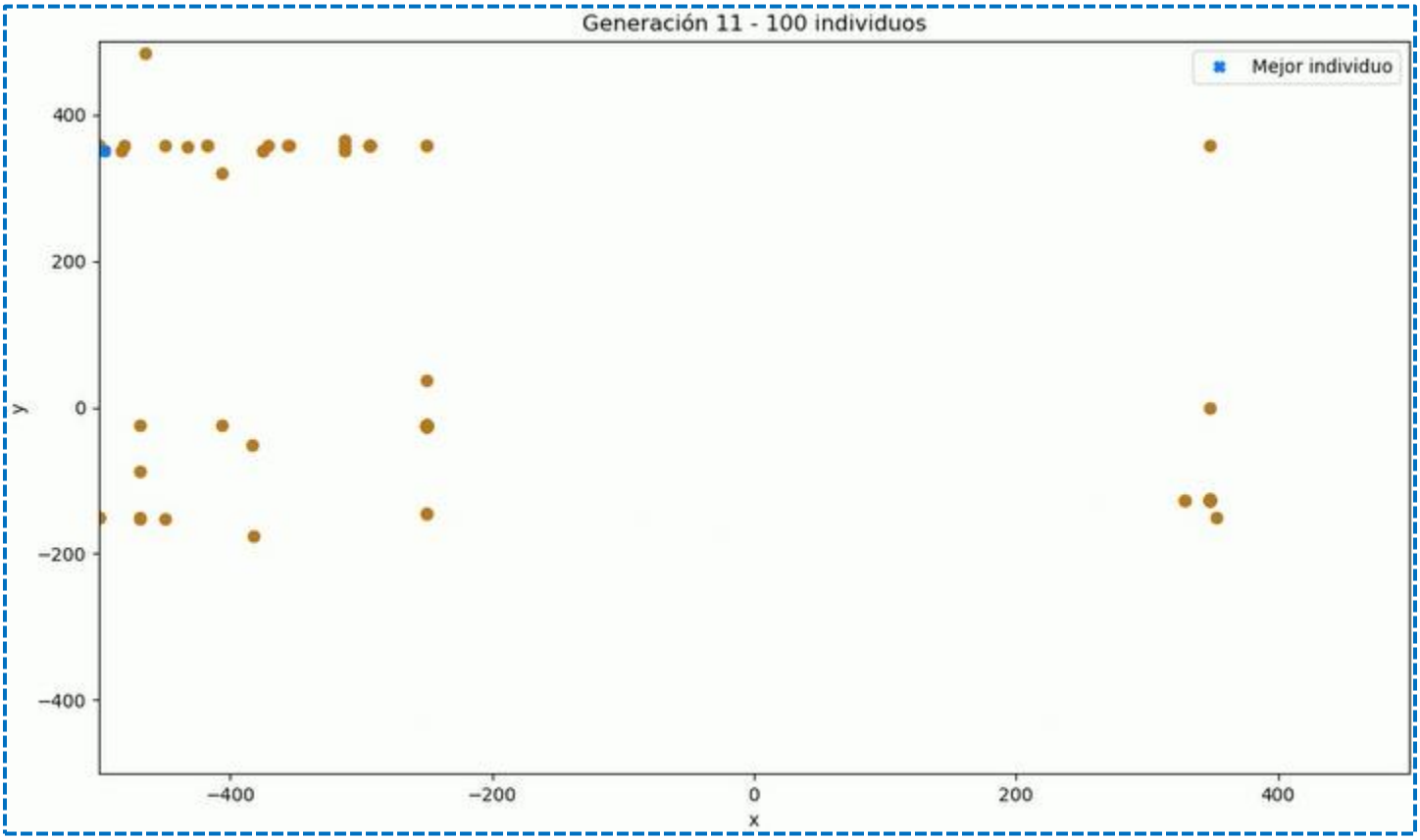


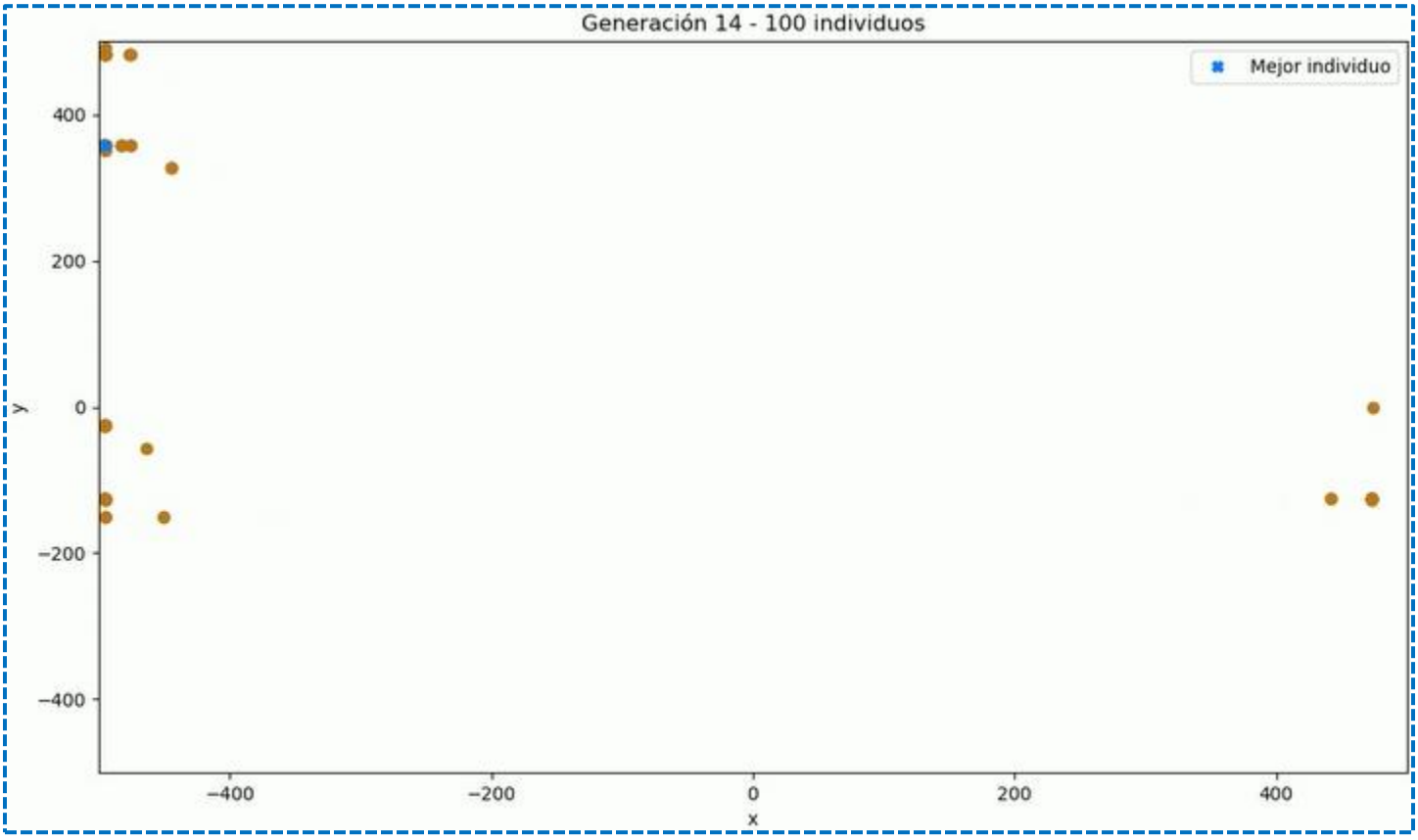
3.1 Video que muestra la evolución de la población.

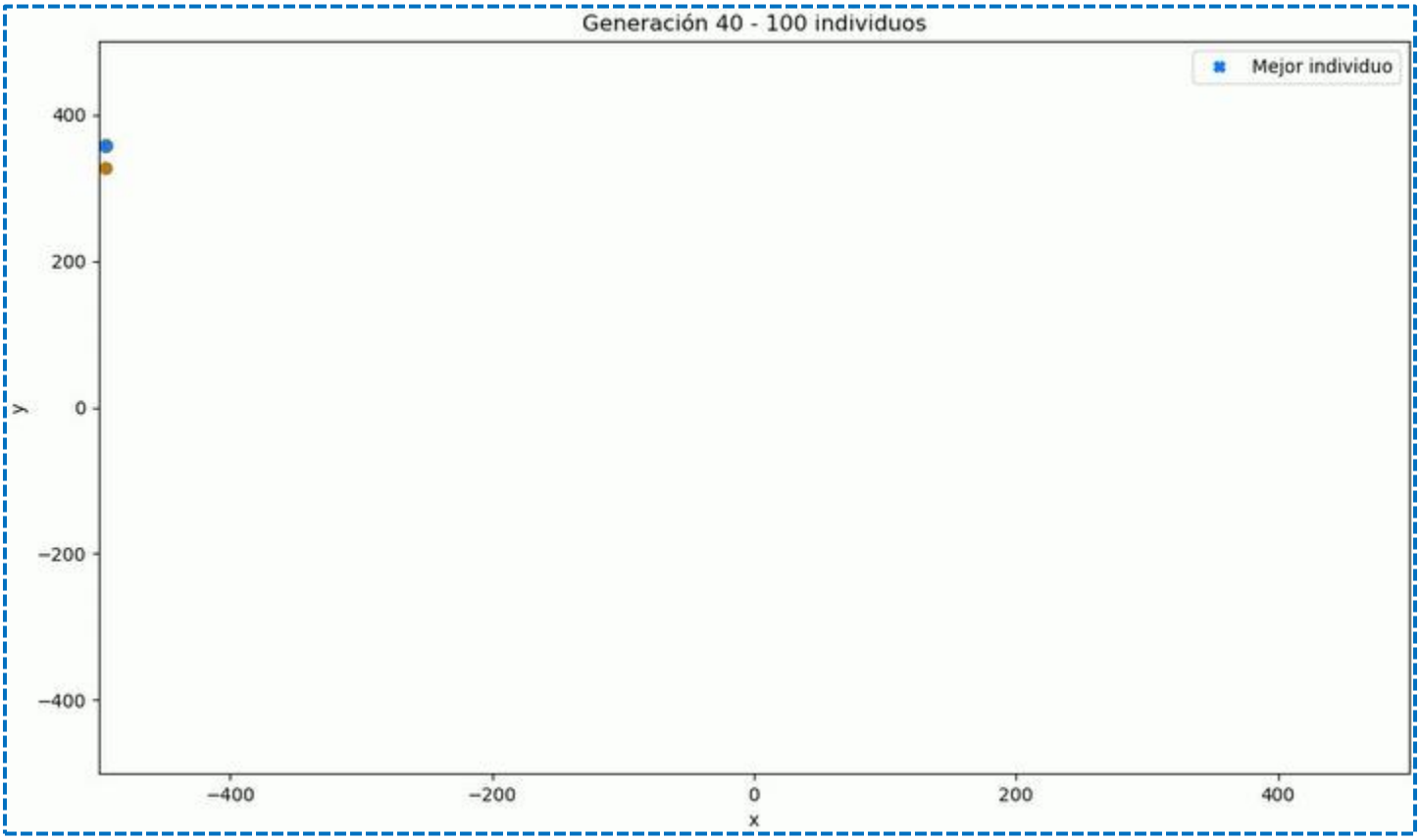
A continuación, se presentan los fotogramas del video, correspondientes a las generaciones más interesantes y la última.

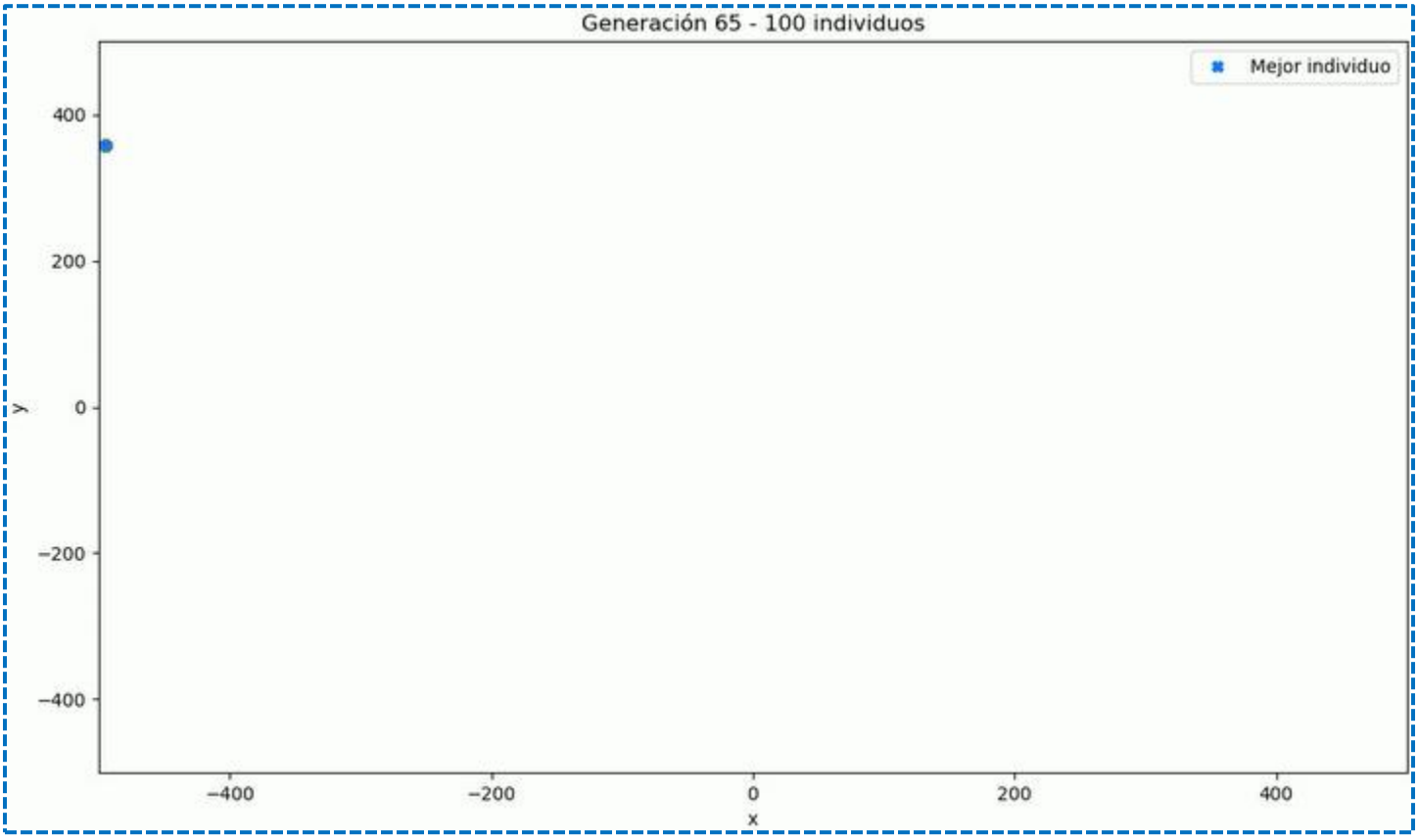

Generación inicial - 4 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x


Generación 8 - 100 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x

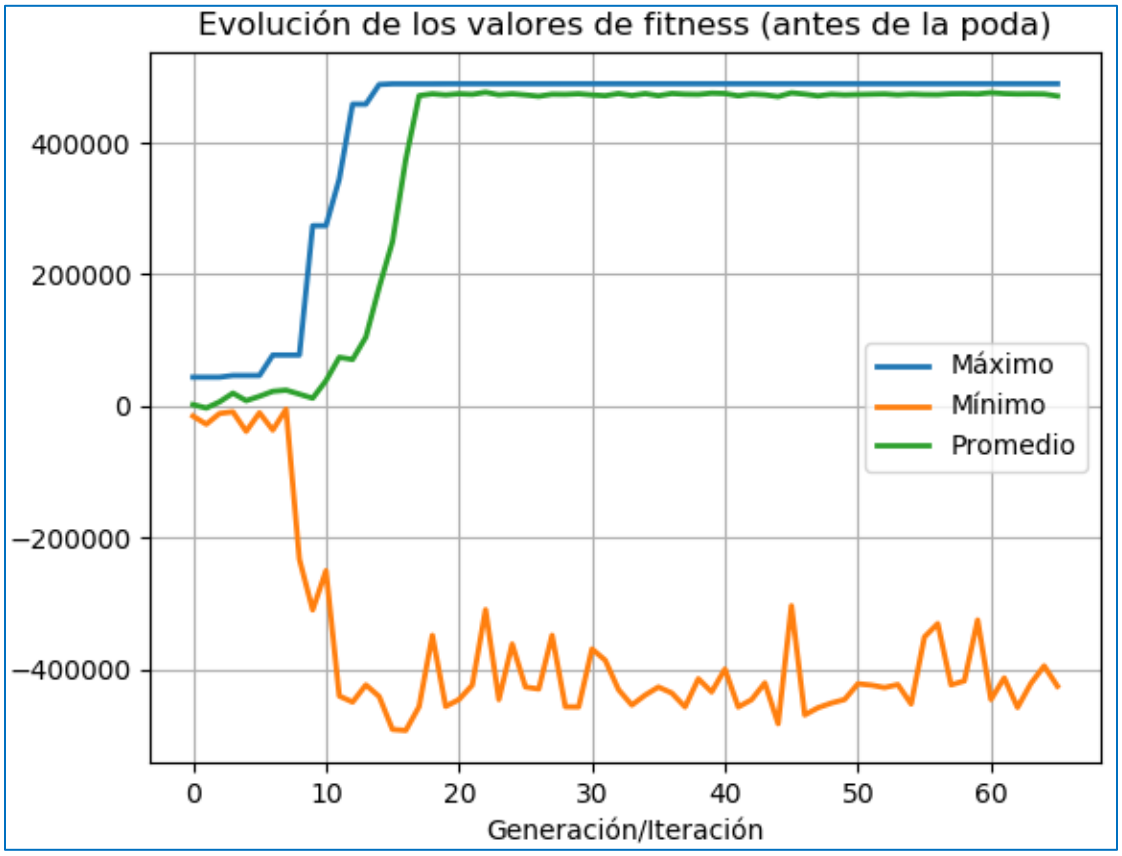

Generación 11 - 100 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x


Generación 14 - 100 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x


Generación 40 - 100 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x


Generación 65 - 100 individuos
Mejor individuo
400
200
0
−200
−400
y
−400
−200
0
200
400
x

3.2 Gráfica donde se visualiza la evolución de la media, el mejor caso y el peor caso para todas las generaciones



4 Discusión

En el video se puede apreciar lo siguiente:

1. En los primeros fotogramas, se aprecia que los puntos correspondientes a los individuos, están bastante dispersos.
2. Algunos individuos se mueven hacia la mitad izquierda y otros, a la mitad derecha.
3. Todos los individuos de la mitad derecha terminan muriendo; mientras tanto, los de la mitad izquierda se van pegando al borde izquierdo.
4. Conforme pasan las generaciones, los individuos se van uniendo más y más, hacia el borde derecho y hacia la mitad inferior.
5. En la generación 16 sólo quedan entre 2 grupos de individuos (o concentraciones) que están muy cerca los unos de los otros.
6. Conforme pasan las generaciones (después de la 16), a veces sale un nuevo grupo, otro se elimina; de tal modo que hay de entre 1 y 4 grupos en el resto de generaciones.
7. En la generación final, sólo queda un grupo, en la que al menos el 60% de los individuos tiene el mejor fitness de todo el procedimiento y, por lo tanto, se considera su fenotipo como la mejor solución al problema (maximizar la función).

Un grupo de puntos se forma por las siguientes razones:

- Todos los puntos que conforman el grupo son iguales.
- Los puntos están tan cerca los unos de los otros que, viéndolos sin acercamiento, pareciese que son iguales.
- En el grupo hay puntos iguales y puntos muy similares.

Observando la gráfica, nos podemos percatar de que las generaciones que más mejoras presentan respecto a sus predecesoras, fueron las primeras 16.

Desde la décima generación se observa una gran brecha ente el mejor (el máximo) y el peor caso (el mínimo). Los peores casos salen de hijos que no son tan buenos como los padres y de mutaciones que no los favorecieron. Los peores casos de cada generación siempre andan variando; a diferencia de los mejores, que casi no presentan variaciones a partir de la treceava generación.

En varias generaciones, el mejor caso es el mismo que sus predecesoras; no fue hasta en la generación 65, que el 60% de los individuos fueron iguales y tenían el mismo mejor valor. Gracias a lo último, ya no se realizaron las iteraciones que darían lugar de la generación 66, a la 100.

Se pudo observar que la cantidad de individuos por generación creció demasiado rápido, alcanzando el límite en la octava:

Generación	1 a 3	4 y 5	6	7	8 a 65
Individuos	4	6	10	22	100

Con esto, se puede reconocer la importancia del límite de individuos ya que, sin éste, la población podría llegar a 1000, 10000, 100000, 1000000 o más individuos, lo que supondría un gran coste de recursos y tiempo.

En conclusión, los algoritmos genéticos dan resultados bastante buenos, pero considero que hacer uso de ellos tiene cierto grado de dificultad por las siguientes razones:

- El código debe de ser adaptado a la cantidad de variables de la función fitnesss que vayamos a usar. O bien, se podría hacer una versión alternativa del código, que acepte como parámetro, dicha cantidad.
- Es necesario hacer un análisis profundo del problema que en el que se esté trabajando, para crear la función fitness.

Se manejaron cadenas de 32 bits, lo que da pie a 4,294,967,295 soluciones posibles; sería una labor titánica probarlos todos a mano o con un algoritmo de búsqueda de fuerza bruta. Los algoritmos genéticos nos dan resultados que, aunque no sean exactamente los verdaderos, están bastante cerca y, por lo tanto, son aceptables.

5 Bibliografía

1. Universidad del País Vasco. (s.f.). *Algoritmos Genéticos.* Recuperado el 14 de Febrero de 2019, de http://www.sc.ehu.es/ccwbayes/docencia/mmcc/docs/temageneticos.pdf